

TurtleBot3 SBC Setup



Contents

1 SBC에 Ubuntu24.04 환경 구축하기

1.1 Raspberry Pi Imager 설치

1.2 Raspberry Pi Imager 실행

1.3 Raspberry Pi Device → Raspberry Pi 4 선택

1.4 CHOOSE OS → Other general-purpose OS → Ubuntu Ubuntu Server (64-bit) 선택

1.5 CHOOSE STORAGE에서 삽입한 microSD 카드 선택

1.6 NEXT 클릭 후 EDIT SETTINGS 클릭

1.7 Username, Password, Wi-Fi 설정

1.8 SERVICES 탭에서 SSH 활성화 후 SAVE

1.9 YES

1.10 YES 클릭 후 PC 비밀번호 입력 후 sd 카드 굽기 시작

1.11 완료 되었다면

1.12 라즈베리파이에 microSD 삽입

1.13 배터리 연결하고, OpenCR과 라즈베리파이 전원 연결 확인

1.14 [SBC] 로봇이 할당받은 IP 주소 확인하기

1.15 [SBC] 만약에 공유기와 연결이 안 되어서 IP 주소가 안뜨는 경우 설정파일 열어서 적용

1.16 [SBC] 변경 적용 후 다시 확인하기

1.17 [PC] PC에서 로봇으로 원격 접속하기

1.18 [SBC] 우선 자동 업데이트 끄기

1.19 [SBC] 부팅 지연 방지 설정

1.20 [SBC] 일시 중단 및 최대 절전 모드 사용 안 함으로 설정

2 SBC에 ROS2 환경 구축하기

2.1 ROS2 Jazzy 설치 페이지

2.2 설치 과정 따라가기

2.3 [SBC] Set localelocale이란?

2.4 [SBC] locale 설정 확인

2.5 [SBC] universe 확인Ubuntu universe 저장소

2.6 [SBC] ROS 2 전용APT 저장소를자동으로등록

2.7 [SBC] apt update

2.8 [SBC] Development tools 설치Development tools란?

2.9 [SBC] apt upgrade 진행

2.10 [SBC] ros-jazzy-ros-base 설치 시작

3 SBC에 터틀봇 환경 구축하기

3.1 [SBC] 터틀봇 패키지의 의존성 패키지들 설치

3.2 [SBC] 터틀봇 패키지 설치

3.3 [SBC] 빌드

3.4 [SBC] bashrc 에 추가

3.5 [SBC] OpenCR 연결용 udev 규칙 설정

3.6 [SBC] ROS Domain ID 설정

3.7 [SBC] LDS 모델 설정

3.8 참고

4 OpenCR 펌웨어 설정

4.1 [SBC] dependency 설치

4.2 [SBC] 환경 변수 설정

4.3 [SBC] 펌웨어 다운로드 후 압축 풀기

4.4 [SBC] 펌웨어 업로드 하면 완료

5 **터틀봇 동작 테스트**

5.1 [SBC] 로봇에서 bringup 실행

5.2 [SBC] 아래와 같이 run 메시지가 출력되면 bringup 완료

5.3 [PC] PC에서 터틀봇 teleop 노드 실행

TurtleBot3 SBC Setup

Exported from Confluence · <https://pinkwink.atlassian.net>

SBC에 Ubuntu24.04 환경 구축하기

Raspberry Pi Imager 설치

Code

```
1 sudo apt update
2 sudo apt install rpi-imager
```

```
mkh@mkh:~$ sudo apt install rpi-imager
[sudo: authenticate] Password:
Installing:
  rpi-imager
```

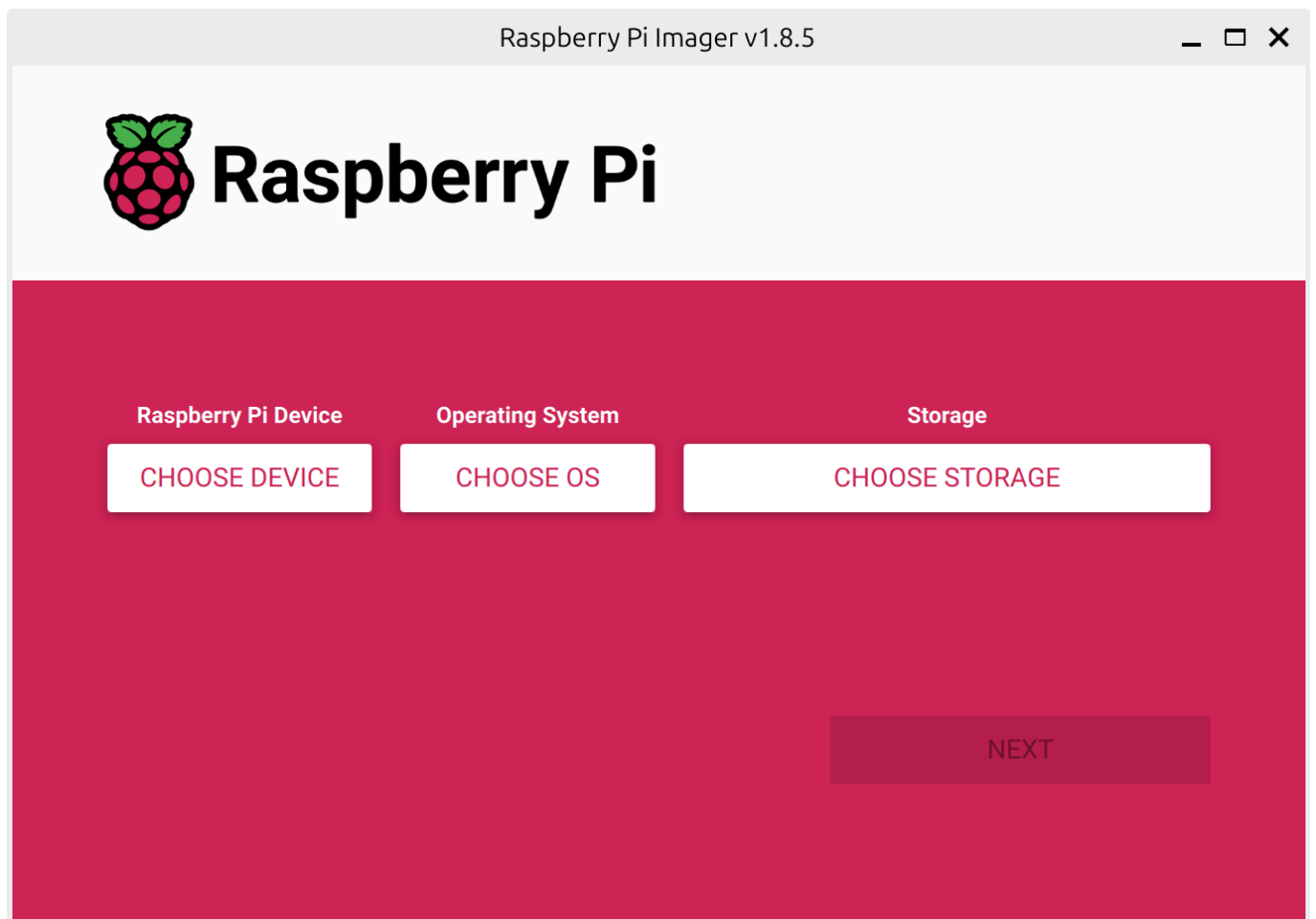
Installing dependencies:

```
javascript-common      libqt5waylandclient5
libdouble-conversion3  libqt5waylandcompositor5
```

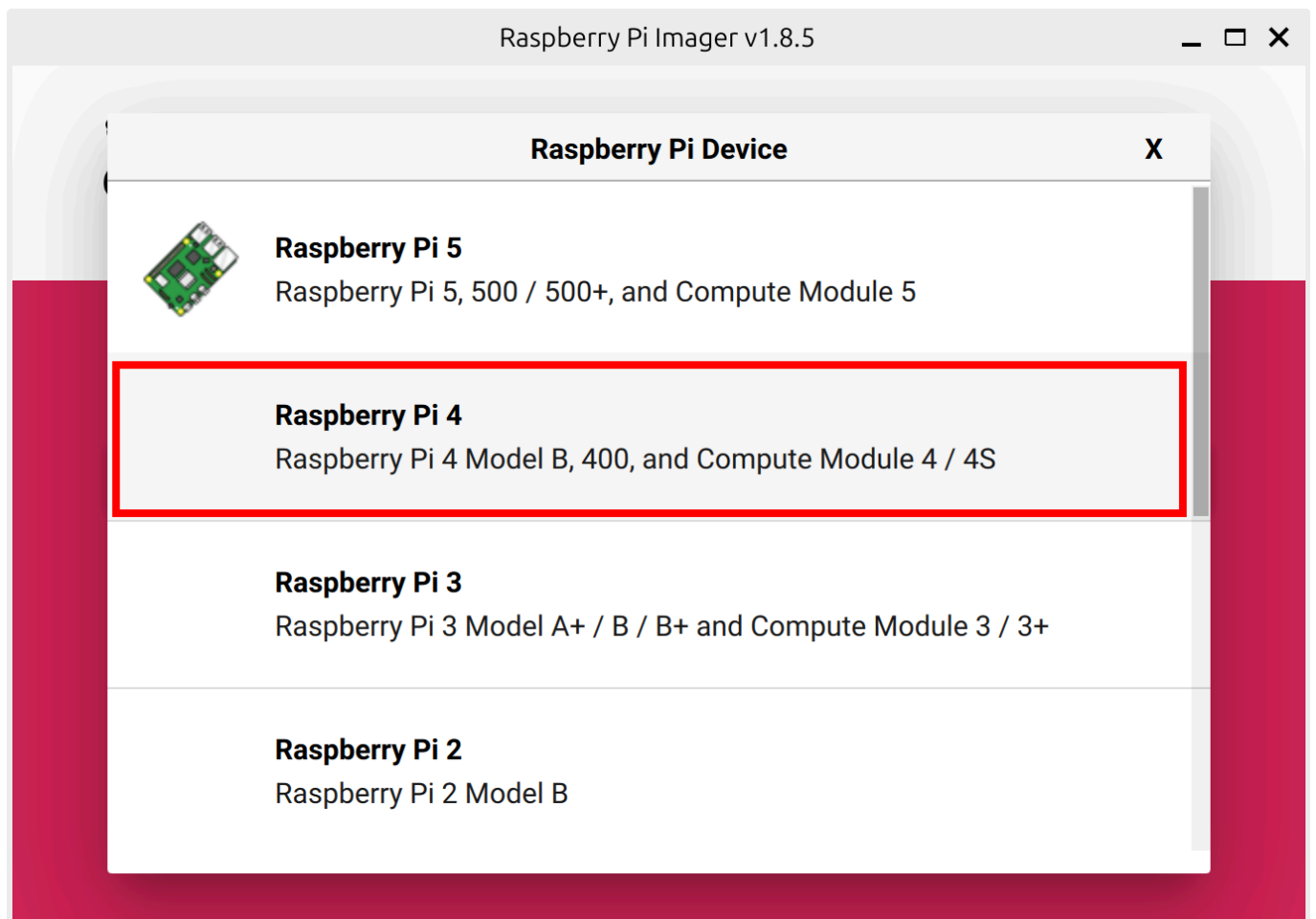
Raspberry Pi Imager 실행

Code

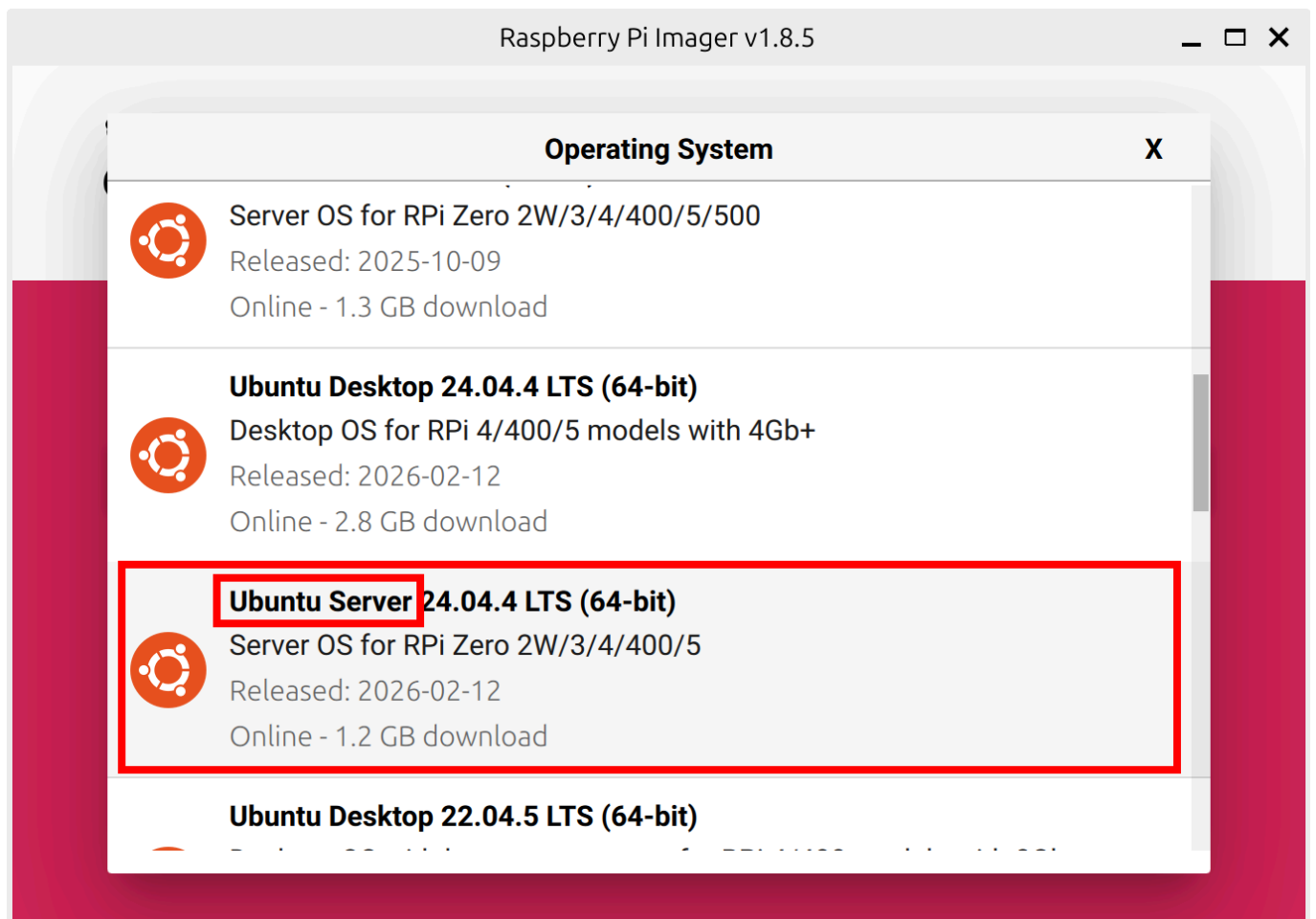
```
1 rpi-imager
```



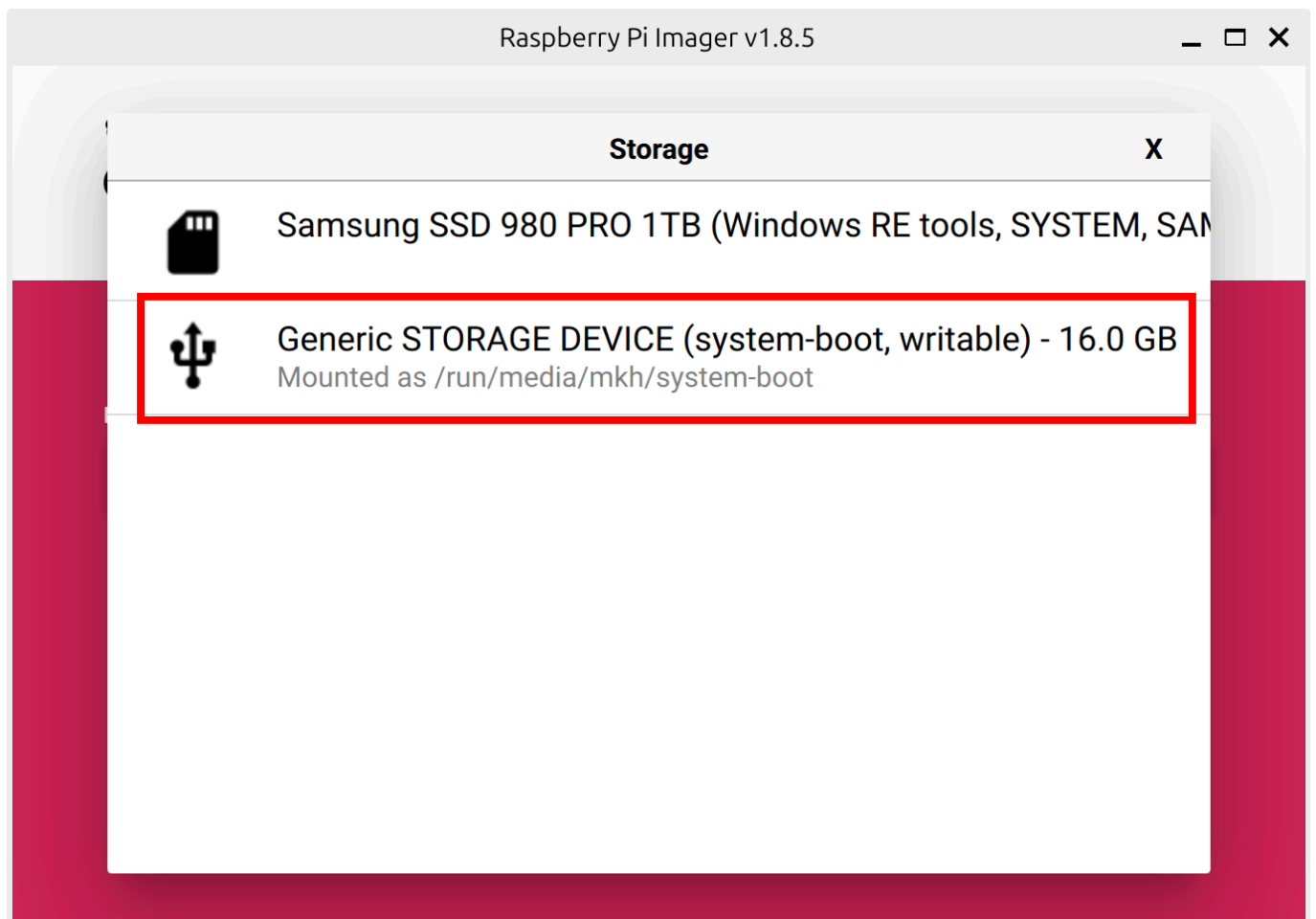
Raspberry Pi Device → Raspberry Pi 4 선택



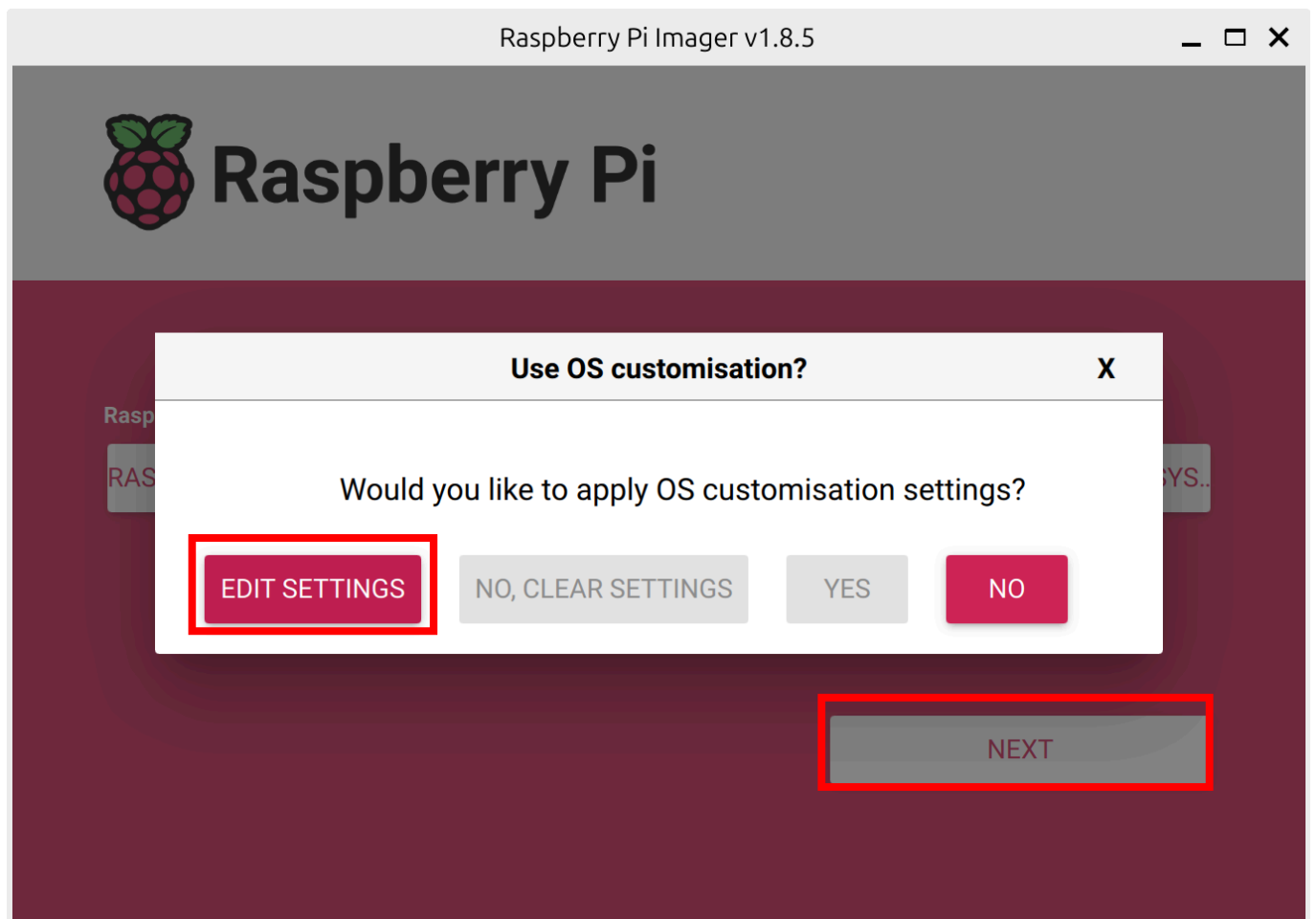
CHOOSE OS → Other general-purpose OS → Ubuntu Ubuntu Server (64-bit) 선택



CHOOSE STORAGE에서 삽입한 microSD 카드 선택



NEXT 클릭 후 EDIT SETTINGS 클릭



Username, Password, Wi-Fi 설정

OS Customisation

GENERAL

SERVICES

OPTIONS

☐

Set hostname: raspberrypi.local

☒

Set username and password

Username: ubuntu

Password:

☒

Configure wireless LAN

SSID: PinkLAB

Password:

☐ Show password

☐ Hidden SSID

Wireless LAN country: GB

☐

Set locale settings

Time zone: Asia/Seoul

Keyboard layout: us

SAVE

SERVICES 탭에서 SSH 활성화 후 SAVE

OS Customisation

GENERAL

SERVICES

OPTIONS

☒ Enable SSH

☒ Use password authentication

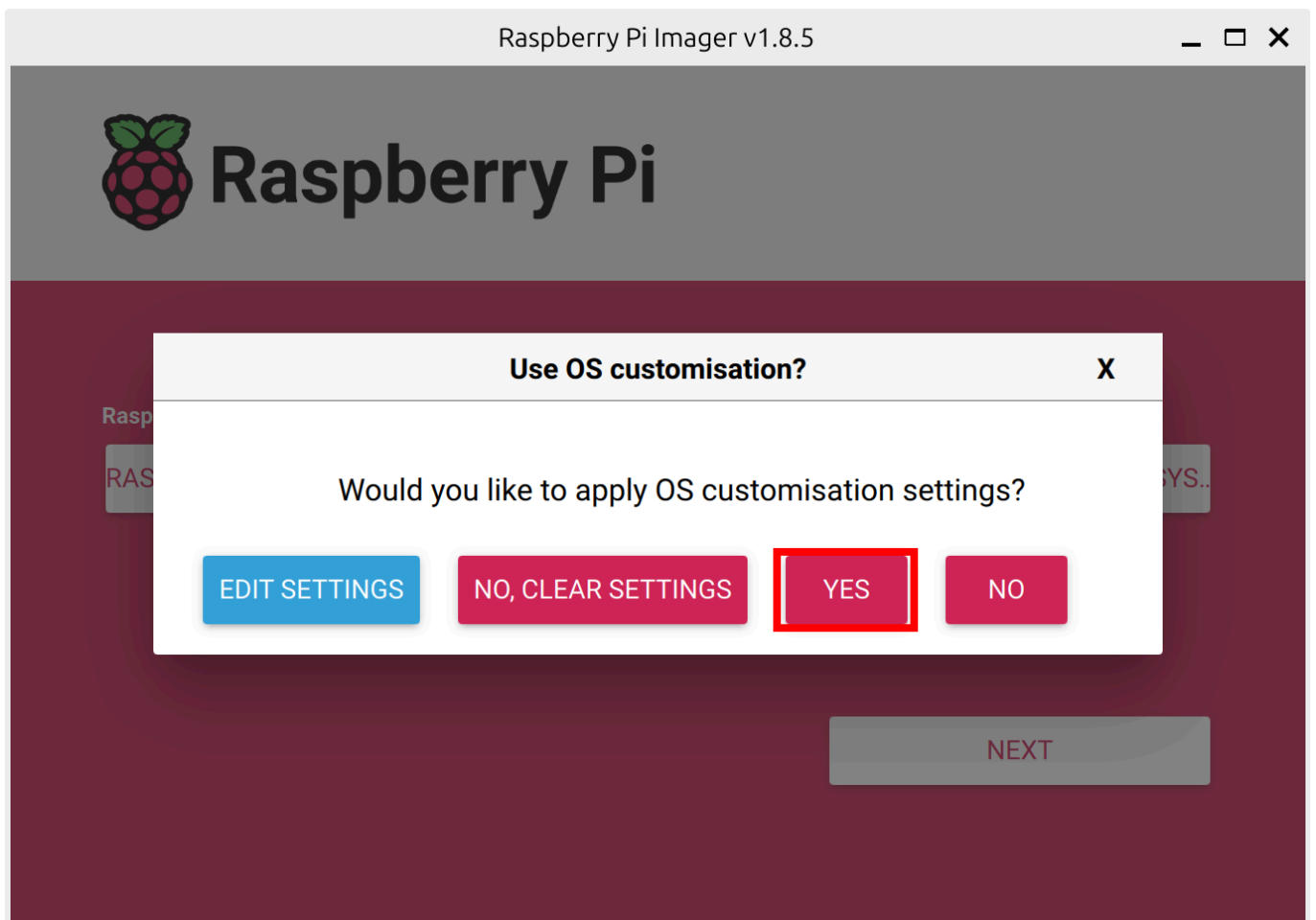
☐ Allow public-key authentication only

Set authorized_keys for 'ubuntu':

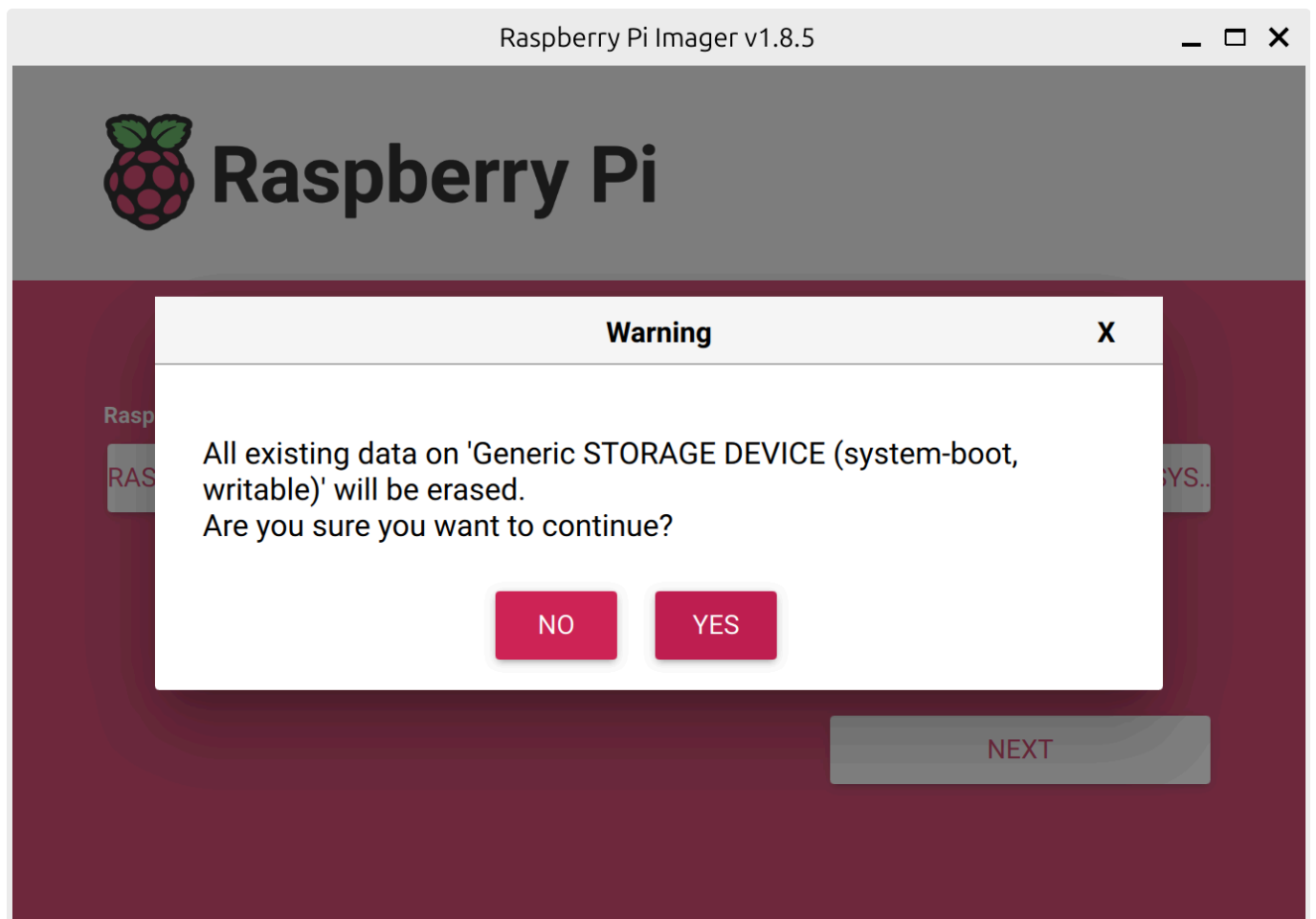
RUN SSH-KEYGEN

SAVE

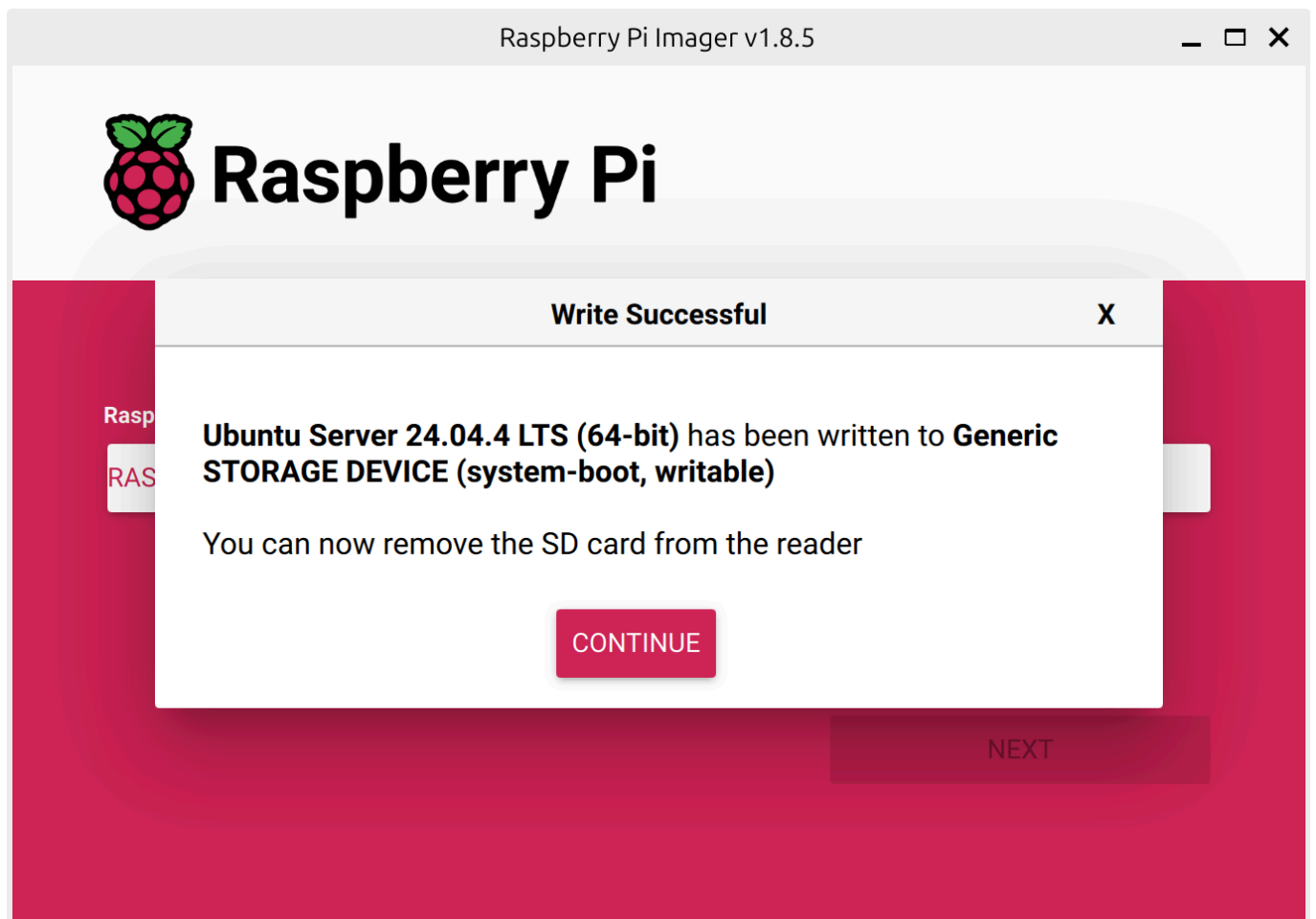
YES



YES 클릭 후 PC 비밀번호 입력 후 sd 카드 굽기 시작

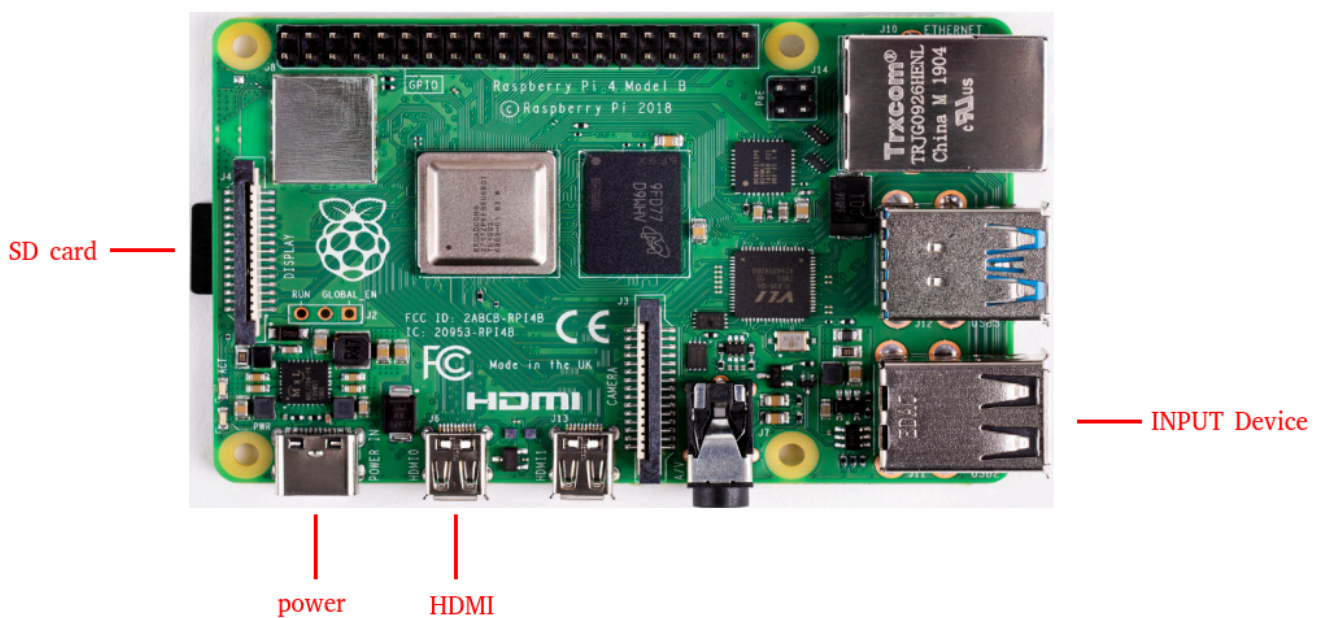


완료 되었다면



라즈베리파이에 microSD 삽입

HDMI, 키보드, 마우스 연결



배터리 연결하고, OpenCR과 라즈베리파이 전원 연결 확인

[SBC] 로봇이 할당받은 IP 주소 확인하기

Code

```
1 ip a
```

```
ubuntu@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether d8:3a:dd:22:dc:fb brd ff:ff:ff:ff:ff:ff
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d8:3a:dd:22:dc:fc brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.12/24 metric 600 brd 192.168.0.255 scope global dynamic wlan0
        valid_lft 7186sec preferred_lft 7186sec
    inet6 fe80::da3a:ddff:fe22:dcfc/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu:~$
```

[SBC] 만약에 공유기와 연결이 안 되어서 IP 주소가 안뜨는 경우 설정파일 열어서 적용

Code

```
1 sudo nano /etc/netplan/50-cloud-init.yaml
```



```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml *
network:
  version: 2
  wifis:
    wlan0:
      dhcp4: true
      access-points:
        "와이파이_이름 ":
          password: "비밀번호"
```

[Cancelled]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify	^/ Go To Line

[SBC] 변경 적용 후 다시 확인하기

```
Code
```

```
1 sudo netplan apply
```

```
ubuntu@ubuntu:~$ sudo nano /etc/netplan/50-cloud-init.yaml
ubuntu@ubuntu:~$ sudo netplan apply
ubuntu@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default ql
en 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 100
0
    link/ether d8:3a:dd:22:dc:fb brd ff:ff:ff:ff:ff:ff
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group d
efault qlen 1000
    link/ether d8:3a:dd:22:dc:fc brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.12/24 metric 600 brd 192.168.0.255 scope global dynamic wlan0
        valid_lft 7183sec preferred_lft 7183sec
    inet6 fe80::da3a:ddff:fe22:dcfc/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu:~$
```

[PC] PC에서 로봇으로 원격 접속하기

Code

```
1  ssh <username>@<로봇의 ip>
```

- yes 입력 후 SBC의 암호 입력

```
mkh@mkh:~$ ssh ubuntu@192.168.0.12
```

The authenticity of host '192.168.0.12 (192.168.0.12)' can't be established.

ED25519 key fingerprint is: SHA256:wfQ7Ie6wuamIf7a7lpZNEcTv9f+kV6hhgVG4/XdrDAs

This key is not known by any other names.

Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

Warning: Permanently added '192.168.0.12' (ED25519) to the list of known hosts.

ubuntu@192.168.0.12's password: █

[SBC] 우선 자동 업데이트 끄기

Code

```
1 sudo nano /etc/apt/apt.conf.d/20auto-upgrades
```

```
GNU nano 7.2 /etc/apt/apt.conf.d/20auto-upgrades *
APT::Periodic::Update-Package-Lists "0";
APT::Periodic::Unattended-Upgrade "0";

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

[SBC] 부팅 지연 방지 설정

네트워크 없는 경우에도 부팅 지연을 방지하도록 설정

Code

```
1 sudo systemctl mask systemd-networkd-wait-online.service
```

[SBC] 일시 중단 및 최대 절전 모드 사용 안 함으로 설정

Code

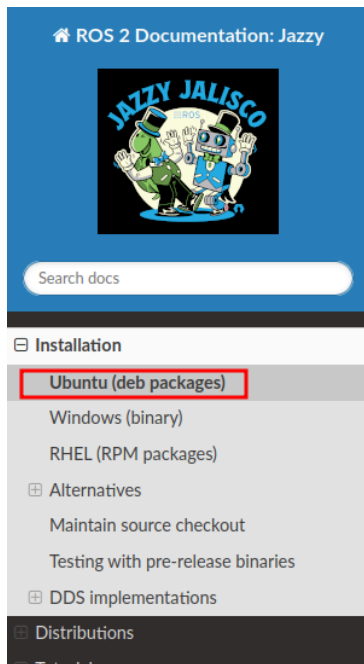
```
1 sudo systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
```

SBC에 ROS2 환경 구축하기

ROS2 Jazzy 설치 페이지

<https://docs.ros.org/en/jazzy/index.html> (<https://docs.ros.org/en/jazzy/index.html>)

- Ubuntu (deb packages) 선택



🏠 / Installation / Ubuntu (deb packages)

[Edit on GitHub](#)

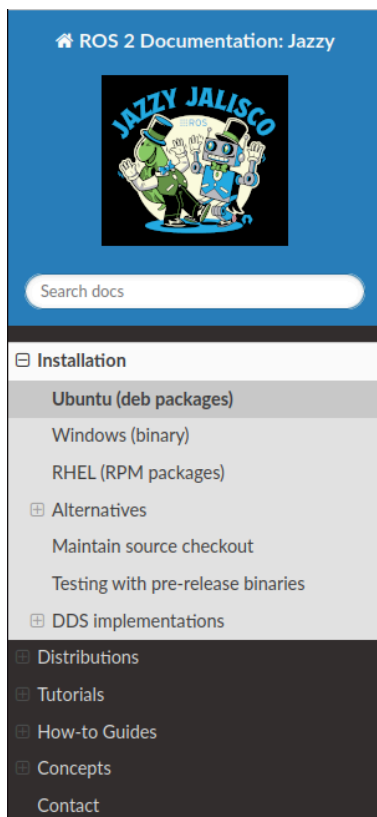
Ubuntu (deb packages)

Table of Contents

- [Resources](#)
- [System setup](#)
 - [Set locale](#)
 - [Enable required repositories](#)
 - [Install development tools \(optional\)](#)
- [Install ROS 2](#)
 - [Install additional RMW implementations \(optional\)](#)
- [Setup environment](#)
- [Try some examples](#)
- [Next steps](#)
- [Troubleshoot](#)
- [Uninstall](#)

Deb packages for ROS 2 Jazzy Jalisco are currently available for Ubuntu Noble (24.04). The target platforms are defined in [RFP 2000](#)

설치 과정 따라가기



System setup

Set locale

Make sure you have a locale which supports `UTF-8`. If you are in a minimal environment (such as a docker container), the locale may be something minimal like `POSIX`. We test with the following settings. However, it should be fine if you're using a different UTF-8 supported locale.

```
locale # check for UTF-8

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

locale # verify settings
```

Enable required repositories

You will need to add the ROS 2 apt repository to your system.

First ensure that the [Ubuntu Universe repository](#) is enabled.

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

Now add the ROS 2 GPG key with apt.

[SBC] Set locale

- **locale**: 시스템의 언어, 문자 인코딩, 날짜/시간 형식 등을 정의하는 설정
- ROS2는 **UTF-8** 인코딩을 요구하므로 locale 설정이 필수
- Copy 해서 붙여넣기

Set locale

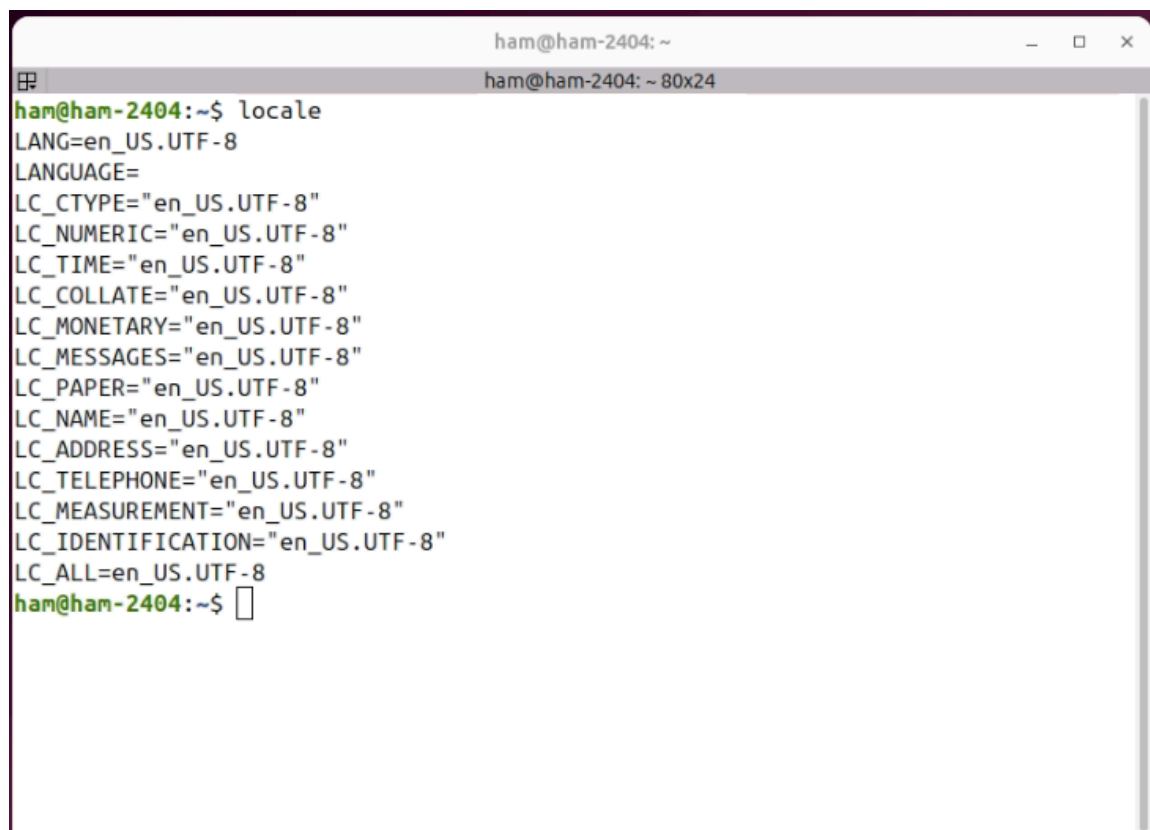
Make sure you have a locale which supports `UTF-8`. If you are in a minimal environment (such as a docker container), the locale may be something minimal like `POSIX`. We test with the following settings. However, it should be fine if you're using a different UTF-8 supported locale.

```
locale # check for UTF-8

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

locale # verify settings
```

[SBC] locale 설정 확인

A terminal window titled 'ham@ham-2404: ~' showing the output of the 'locale' command. The output lists various locale settings, all of which are configured to 'en_US.UTF-8'.

```
ham@ham-2404:~$ locale
LANG=en_US.UTF-8
LANGUAGE=
LC_CTYPE="en_US.UTF-8"
LC_NUMERIC="en_US.UTF-8"
LC_TIME="en_US.UTF-8"
LC_COLLATE="en_US.UTF-8"
LC_MONETARY="en_US.UTF-8"
LC_MESSAGES="en_US.UTF-8"
LC_PAPER="en_US.UTF-8"
LC_NAME="en_US.UTF-8"
LC_ADDRESS="en_US.UTF-8"
LC_TELEPHONE="en_US.UTF-8"
LC_MEASUREMENT="en_US.UTF-8"
LC_IDENTIFICATION="en_US.UTF-8"
LC_ALL=en_US.UTF-8
ham@ham-2404:~$
```

[SBC] universe 확인

- **universe:** Ubuntu의 커뮤니티 관리 오픈소스 패키지 저장소
- ROS2 패키지가 이 저장소를 통해 배포되므로 활성화 필요

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

[SBC] ROS 2 전용 APT 저장소를 자동으로 등록

```
$ sudo apt update && sudo apt install curl -y
$ export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest | grep -F "tag_n
$ curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/r
$ sudo apt install /tmp/ros2-apt-source.deb
```

[SBC] apt update

새로 추가한 ROS2 저장소의 패키지 목록을 갱신합니다.

```
jt@ubuntu:~$ sudo apt update
Hit:1 https://packages.microsoft.com/repos/code stable InRelease
Hit:2 https://dl.google.com/linux/chrome/deb stable InRelease
Hit:3 http://kr.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://kr.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:6 http://packages.ros.org/ros2/ubuntu noble InRelease [4,676 B]
Hit:7 http://kr.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:8 http://packages.ros.org/ros2/ubuntu noble/main amd64 Packages [1,060 kB]
Fetched 1,064 kB in 3s (367 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

[SBC] Development tools 설치

- ROS2 패키지를 소스에서 빌드할 때 필요한 도구 모음
- **colcon**(빌드 도구), **rosdep**(의존성 관리), **vcstool**(버전 관리) 등 포함

```
jt@ubuntu:~$ sudo apt update && sudo apt install ros-dev-tools
Hit:1 https://packages.microsoft.com/repos/code stable InRelease
Hit:2 https://dl.google.com/linux/chrome/deb stable InRelease
Hit:3 http://packages.ros.org/ros2/ubuntu noble InRelease
Hit:4 http://kr.archive.ubuntu.com/ubuntu noble InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:6 http://kr.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:7 http://kr.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

[SBC] apt upgrade 진행

Code

```
1 sudo apt upgrade
```

[SBC] ros-jazzy-ros-base 설치 시작

라즈베리파이에 서버용 Ubuntu OS를 설치했기 때문에 GUI 관련 툴이 필요 없다.

ROS-Base Install (Bare Bones): Communication libraries, message packages, command line tools. No GUI tools.

```
$ sudo apt install ros-jazzy-ros-base
```

SBC에 터틀봇 환경 구축하기

[SBC] 터틀봇 패키지의 의존성 패키지들 설치

Code

```
1 sudo apt install python3-argcomplete python3-colcon-common-extensions libboost-system-dev build-essential
2 sudo apt install ros-jazzy-hls-lfcd-lds-driver ros-jazzy-turtlebot3-msgs ros-jazzy-dynixel-sdk ros-jazzy-xacro
3 sudo apt install libudev-dev
```

[SBC] 터틀봇 패키지 설치

Code

```
1 mkdir -p ~/turtlebot3_ws/src && cd ~/turtlebot3_ws/src
2 git clone -b jazzy https://github.com/ROBOTIS-GIT/turtlebot3.git
3 git clone -b jazzy https://github.com/ROBOTIS-GIT/ld08_driver.git
4 git clone -b jazzy https://github.com/ROBOTIS-GIT/coin_d4_driver
5 cd ~/turtlebot3_ws/src/turtlebot3 && rm -r turtlebot3_cartographer turtlebot3_navigation2
```

[SBC] 빌드

Code

```
1 cd ~/turtlebot3_ws/
2 echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
3 source ~/.bashrc
4 colcon build --symlink-install --parallel-workers 1
```

[SBC] bashrc 에 추가

Code

```
1 echo 'source ~/turtlebot3_ws/install/setup.bash' >> ~/.bashrc
2 source ~/.bashrc
```

[SBC] OpenCR 연결용 udev 규칙 설정

Code

```
1 sudo cp `ros2 pkg prefix turtlebot3_bringup`/share/turtlebot3_bringup/script/99-turtlebot3-cdc.rules /etc/udev/rules.d/
2 sudo udevadm control --reload-rules
3 sudo udevadm trigger
```

[SBC] ROS Domain ID 설정

- pc 와 같은 ROS_DOMAIN_ID 설정

Code

```
1 echo 'export ROS_DOMAIN_ID=30' >> ~/.bashrc
2 source ~/.bashrc
```

[SBC] LDS 모델 설정

보유한 LDS 모델에 맞게 환경변수를 설정

Code

```
1 # 사용 모델에 맞춰 하나만 설정
2 echo 'export LDS_MODEL=LDS-01' >> ~/.bashrc
3 # 또는
4 echo 'export LDS_MODEL=LDS-02' >> ~/.bashrc
5 # 또는
6 echo 'export LDS_MODEL=LDS-03' >> ~/.bashrc
7 source ~/.bashrc
```

참고



OpenCR 펌웨어 설정

[SBC] dependency 설치

Code

```
1 sudo dpkg --add-architecture armhf
2 sudo apt-get update
3 sudo apt-get install libc6:armhf
```

[SBC] 환경 변수 설정

가지고 있는 모델이 와플인 경우 burger 대신 waffle 로 변경

Code

```
1 echo "export TURTLEBOT3_MODEL=burger" >> ~/.bashrc
2 echo "export OPENCR_MODEL=burger" >> ~/.bashrc
3 echo "export OPENCR_PORT=/dev/ttyACM0" >> ~/.bashrc
4 source ~/.bashrc
```

[SBC] 펌웨어 다운로드 후 압축 풀기

Code

```
1 wget https://github.com/ROBOTIS-GIT/OpenCR-Binaries/raw/master/turtlebot3/ROS2/latest/opencr_update.tar.bz2
2 tar -xvf opencr_update.tar.bz2
```

[SBC] 펌웨어 업로드 하면 완료

Code

```
1 cd ./opencr_update
2 ./update.sh $OPENCNCR_PORT $OPENCNCR_MODEL.opencr
```

```
ubuntu@ubuntu:~/opencr_update$ cd ./opencr_update
./update.sh $OPENCR_PORT $OPENCR_MODEL.opencr
-bash: cd: ./opencr_update: No such file or directory
aarch64
arm
OpenCR Update Start..
opencr_ld_shell ver 1.0.0
opencr_ld_main
[ ] file name      : burger.opencr
[ ] file size     : 136 KB
[ ] fw_name       : burger
[ ] fw_ver        : V230127R1
[OK] Open port    : /dev/ttyACM0
[ ]
[ ] Board Name    : OpenCR R1.0
[ ] Board Ver     : 0x17020800
[ ] Board Rev     : 0x00000000
[OK] flash_erase  : 1.12s
[OK] flash_write  : 1.35s
[OK] CRC Check    : D92222 D92222 , 0.004000 sec
[OK] Download
[OK] jump_to_fw
ubuntu@ubuntu:~/opencr_update$ █
```

터틀봇 동작 테스트

[SBC] 로봇에서 bringup 실행

Code

```
1  ros2 launch turtlebot3_bringup robot.launch.py
```

```

ubuntu@desktop:~$ ros2 launch turtlebot3_bringup robot.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-08-06-15-38-14-170869-desktop-1899
[INFO] [launch]: Default logging verbosity is set to INFO
urdf_file_name : turtlebot3_waffle_pi.urdf
[INFO] [robot_state_publisher-1]: process started with pid [1900]
[INFO] [ld08_driver-2]: process started with pid [1902]
[INFO] [turtlebot3_ros-3]: process started with pid [1904]
[turtlebot3_ros-3] [INFO] [1722926295.300915205] [turtlebot3_node]: Init TurtleBot3 Node Main
[turtlebot3_ros-3] [INFO] [1722926295.302007094] [turtlebot3_node]: Init DynamixelSDKWrapper
[turtlebot3_ros-3] [INFO] [1722926295.309993779] [DynamixelSDKWrapper]: Succeeded to open the port(/dev/ttyACM0)!
[turtlebot3_ros-3] [INFO] [1722926295.318406890] [DynamixelSDKWrapper]: Succeeded to change the baudrate!
[robot_state_publisher-1] [INFO] [1722926295.355558631] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-1] [INFO] [1722926295.355946112] [robot_state_publisher]: got segment base_link
[robot_state_publisher-1] [INFO] [1722926295.356014019] [robot_state_publisher]: got segment base_scan
[robot_state_publisher-1] [INFO] [1722926295.356049668] [robot_state_publisher]: got segment camera_link

```

[SBC] 아래와 같이 run 메시지가 출력되면 bringup 완료

```

[turtlebot3_ros-3] [INFO] [1723012399.729643973] [turtlebot3_node]: Calibration End
[turtlebot3_ros-3] [INFO] [1723012399.729768640] [turtlebot3_node]: Add Motors
[turtlebot3_ros-3] [INFO] [1723012399.730313087] [turtlebot3_node]: Add Wheels
[turtlebot3_ros-3] [INFO] [1723012399.730806015] [turtlebot3_node]: Add Sensors
[turtlebot3_ros-3] [INFO] [1723012399.737617624] [turtlebot3_node]: Succeeded to create battery state publisher
[turtlebot3_ros-3] [INFO] [1723012399.742116776] [turtlebot3_node]: Succeeded to create imu publisher
[turtlebot3_ros-3] [INFO] [1723012399.746027593] [turtlebot3_node]: Succeeded to create sensor state publisher
[turtlebot3_ros-3] [INFO] [1723012399.748445475] [turtlebot3_node]: Succeeded to create joint state publisher
[turtlebot3_ros-3] [INFO] [1723012399.748567494] [turtlebot3_node]: Add Devices
[turtlebot3_ros-3] [INFO] [1723012399.748610032] [turtlebot3_node]: Succeeded to create motor power server
[turtlebot3_ros-3] [INFO] [1723012399.751544528] [turtlebot3_node]: Succeeded to create reset server
[turtlebot3_ros-3] [INFO] [1723012399.754058818] [turtlebot3_node]: Succeeded to create sound server
[turtlebot3_ros-3] [INFO] [1723012399.756624591] [turtlebot3_node]: Run!
[turtlebot3_ros-3] [INFO] [1723012399.797731799] [diff_drive_controller]: Init Odometry
[turtlebot3_ros-3] [INFO] [1723012399.811079440] [diff_drive_controller]: Run!

```

[PC] PC에서 터틀봇 teleop 노드 실행

w a s d x 키입력으로 속도값을 설정해서 주행 테스트

Code

```
1 ros2 run turtlebot3_teleop teleop_keyboard
```

```
mkh@mkh:~$ ros2 run turtlebot3_teleop teleop_keyboard
```

Control Your TurtleBot3!

Moving around:

	w	
a	s	d
	x	

w/x : increase/decrease linear velocity (Burger : ~ 0.22, Waffle and Waffle Pi : ~ 0.26)

a/d : increase/decrease angular velocity (Burger : ~ 2.84, Waffle and Waffle Pi : ~ 1.82)

space key, s : force stop

CTRL-C to quit

```
currently:      linear velocity 0.01      angular velocity 0.0
```



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio